VLSI IMPLEMENTATION OF AN RSA HARDWARE ACCELARATOR USING AMBA AXI4-LITE INTERFACE

A CAPSTONE PROJECT REPORT

Submitted in partial fulfillment of the requirement for the award of the Degree of

BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMMUNICATION ENGINEERING

by

SHIVANGI NANDA (22BEC7078)

Under the Guidance of

Dr. UMAKANTA NANDA

SCHOOL OF ELECTRONICS ENGINEERING
VIT-AP UNIVERSITY
AMARAVATI- 522241

November 2025

CERTIFICATE

This is to certify that the Capstone Project work titled "**VLSI IMPLEMENTATION OF AN RSA HARDWARE ACCELARATOR USING AMBA AXI4-LITE INTERFACE**" that is being submitted by **SHIVANGI NANDA (22BEC7078)** is in partial fulfilment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Dr. UMAKANTA NANDA
Guide

The thesis is satisfactory / unsatisfactory

3/12/25

Internal Examiner-1

Internal Examiner-2

Approved by

PROGRAM CHAIR
B. Tech. ECE

DEAN
School of Electronics Engineering

ACKNOWLEDGEMENTS

I express my sincere and profound gratitude to my respected guide, **Dr. Umakanta Nanda, Professor, School of Electronics Engineering, VIT-AP University**, for his invaluable guidance, continuous encouragement, and unwavering support throughout the entire duration of this project. His insightful suggestions, patient supervision, and constant motivation have been instrumental in shaping the quality and direction of this work. I remain deeply indebted to him for the time and effort he devoted to reviewing and enriching this manuscript despite his demanding academic commitments.

I would also like to extend my heartfelt thanks to **Dr. Bandaru Ramakrishna, Associate Professor**, and **Dr. Samineni Peddakrishna, Associate Professor**, School of Electronics Engineering, VIT-AP University, for being part of my evaluation panel and for their constructive feedback and guidance at various stages of this project.

My sincere appreciation goes to **Dr. Umakanta Nanda, Professor & Dean**, and **Dr. Rohit Lorenzo, Associate Professor**, School of Electronics Engineering, VIT-AP University, for their support and for providing access to the *Chips to Startup (C2S) Lab*, funded by the Government of India, which greatly facilitated the technical execution of this work.

I am also grateful to all the **Ph.D. scholars and non-teaching staff** of the School of Electronics Engineering, VIT-AP University, for their technical assistance, cooperation, and encouragement in numerous aspects during the course of this project.

Above all, I express my deepest gratitude to the **Almighty** for His blessings, guidance, and strength, which have always been a source of inspiration and resilience throughout my academic journey.

Shivangi Nanda

ABSTRACT

Secure data processing has become indispensable in modern embedded systems, IoT platforms, and processor-based System-on-Chip (SoC) architectures. Traditional software-based RSA implementations fail to meet the performance, latency, and energy constraints of real-time applications, motivating the need for dedicated hardware acceleration. This work presents the VLSI design and implementation of a 128-bit RSA hardware accelerator integrated with an AMBA AXI4-Lite interface for seamless SoC integration. The accelerator performs modular exponentiation using an optimized square-and-multiply architecture supported by a Montgomery-based arithmetic engine. A complete AXI4-Lite slave wrapper enables memory-mapped access to configuration registers, operands, keys, and status signals. Designed in Verilog HDL and synthesized using Cadence Genus and Xilinx Vivado, the accelerator achieves functional correctness, protocol-compliant communication, and efficient performance. Timing analysis indicates an achievable operating frequency of 176.68 MHz, with energy results demonstrating a Power-Delay Product (PDP) of 121.4 pJ. The implemented design confirms the practicality of deploying RSA as a high-performance hardware IP in secure SoC platforms and provides a foundation for extensible, low-power cryptographic acceleration.

TABLE OF CONTENTS

S.No.	Chapter	Title	Page Number
1.		Acknowledgement	2
2.		Abstract	3
3.		List of Table	5
4		List of Figures	6
5		List of Symbols	7
4.	1	Introduction	8
	1.1	Objectives	10
	1.2	Background and Literature Survey	11
	1.3	Organization of the Report	13
5.	2	VlSI Implementation of an RSA Hardware Accelerator Using Amba Axi4-Lite Interface	15
	2.1	Proposed System	15
	2.2	Working Methodology	20
	2.3	Standards	23
	2.4	System Details	25
	2.4.1	Software	26
7.	3	Results and Discussions	26
8.	4	Conclusion & Future Works	31
9.	5	Appendix	33
10.	6	References	42

List of Tables

Table No.	Title	Page No.
2.1	Register Map	21
3.1	Performance Metrics	27
3.2	Area and Power Metrics	29

List of Figures

Figure No.	Title	Page No.
2.1	RSA Algorithm Structure	17
2.2	System Architecture	21
2.3	RSA Core Internal Architecture	21
3.1	Simulation Waveform from Vivado	26
3.2	Graphical Representation of Performance Metrics	28
3.3	Graphical Representation of Area Metrics	28
3.4	Graphical Representation of Power Metrics	29

List of Symbols

Symbol	Description
p, q	Large prime numbers used for RSA key generation
N	RSA modulus, computed as $N = p \times q$
ϕ(N)	Euler's Totient of N, calculated as $(p-1)(q-1)$
e	Public exponent used for encryption
d	Private exponent used for decryption
M	Plaintext message
C	Ciphertext output
R	Montgomery radix (power-of-two constant)
t	Intermediate Montgomery multiplication value
CLK	Clock signal
RST	Reset signal
AWADDR	AXI4-Lite write address
WDATA	AXI4-Lite write data
WVALID / WREADY	AXI4-Lite write handshake signals
ARADDR	AXI4-Lite read address
RDATA	AXI4-Lite read data
RVALID / RREADY	AXI4-Lite read handshake signals
CTRL	Control register for start/operation mode
STATUS	Status register (busy/done/error flags)
DIN	Input data register (plaintext or ciphertext)
KEY	Exponent register (public/private key)
N (reg)	RSA modulus register
DOUT	Output data register (result)
WNS	Worst Negative Slack
Tclk	Clock period constraint
tcp	Critical path delay
fmax	Maximum achievable operating frequency
A	Total cell area
P	Total power consumption
Pdyn	Dynamic power component
Pleak	Leakage power component
PDP	Power Delay Product

CHAPTER – 1

INTRODUCTION

Secure communication has become a fundamental requirement in modern electronic systems. With the rapid growth of embedded devices, Internet-of-Things (IoT) platforms, wearable electronics, defense communication modules, and processor-based SoCs, the need to protect sensitive information has never been more critical. As these systems continuously exchange data across wired and wireless networks, hardware-level security mechanisms offer a dependable and high-performance solution for ensuring confidentiality, integrity, and authentication.

Cryptography is the science of securing information through mathematical techniques that prevent unauthorized access or tampering. It protects data during storage, transmission, and processing. In hardware-oriented fields such as embedded systems, IoT security modules, VLSI-based communication SoCs, smart cards, and trusted execution environments, cryptography plays a vital role in preventing device spoofing, securing firmware updates, authenticating devices, and safeguarding private data. Cryptographic algorithms fall into two major categories: **symmetric-key** and **asymmetric-key** cryptography.

Symmetric cryptography uses a single shared key for both encryption and decryption. It is fast and efficient, making it ideal for bulk data encryption, secure memory storage, and on-chip communication. However, its major limitation lies in secure key distribution: the same secret key must be shared between devices, which becomes challenging and risky in large networks.

Asymmetric or public-key cryptography, in contrast, uses a pair of mathematically linked keys—one public and one private. The public key can be openly shared, while the private key remains confidential. Asymmetric algorithms eliminate the key-distribution problem and provide strong authentication and non-repudiation. They are more secure for establishing trust between devices that have never communicated before, making them highly valuable in IoT onboarding, secure boot processes, digital signatures, and encryption key exchanges between embedded systems.

Because of these advantages, **public-key cryptography is better suited than symmetric methods for trust establishment**, secure device identification, and

distributed systems where secure private key sharing is impractical. **Therefore, this project adopts RSA, a widely used public-key cryptographic algorithm, as the foundation of the hardware accelerator design.** This seamlessly connects to the focus of the next section—implementing RSA in hardware.

RSA is one of the most widely used public-key algorithms in secure communication protocols, digital signatures, key exchange, and authentication modules. Its reliability stems from its strong mathematical foundation and long-standing acceptance in industry standards. However, implementing RSA purely in software introduces significant limitations for embedded and real-time systems. Operations such as key processing and modular exponentiation involve large-bit computations, which are slow and resource-heavy on a general-purpose processor. As a result, software implementations struggle to meet the timing, performance, and energy-efficiency requirements of modern applications.

This challenge has created the need for dedicated hardware accelerators, which can execute cryptographic algorithms at much higher speeds while consuming significantly lower energy. By designing RSA at the hardware level, it becomes possible to optimize datapaths, arithmetic logic, and control mechanisms, resulting in improved throughput, predictable latency, and overall system efficiency. Hardware accelerators also ensure consistent execution times, which is crucial for resisting timing-based attacks.

While hardware acceleration solves the performance problem, the next major requirement is seamless integration into today's System-on-Chip (SoC) architectures. SoCs typically combine processors, memory interfaces, peripherals, and custom logic, all connected through a **standardized interconnect protocol**. Among available interconnect standards—such as Wishbone, Avalon, TileLink, and AMBA—**the AMBA (Advanced Microcontroller Bus Architecture)** protocol family from ARM is the most widely adopted in industrial SoC design. Its popularity stems from its **high modularity, extensive tool support, scalability, IP reusability, and strong compatibility with ARM-based processors**, which dominate embedded and IoT platforms.

AMBA includes several protocol variants: **APB** for simple, low-bandwidth peripherals; **AHB** for moderate-speed memory and peripherals; and **AXI/AXI4** for

high-performance systems. Within the AXI family, **AXI4-Lite** is specifically tailored for custom hardware IP blocks that require simple, reliable, and register-level control. AXI4-Lite offers a lightweight, memory-mapped interface, low complexity, independent read/write channels, easy timing closure, and straightforward integration with processor systems like ARM Cortex-M/A and Xilinx Zynq. These qualities make **AXI4-Lite the optimal choice for integrating the RSA accelerator**, as it enables clean register communication without the overhead of burst transfers or complex transaction management.

In this context, the present work focuses on the VLSI design and implementation of an RSA hardware accelerator integrated through an AMBA AXI4-Lite interface. The accelerator is designed using Verilog HDL and mapped to a register-based communication structure that allows a processor to configure the RSA operation, supply input parameters, monitor progress, and retrieve the final result. The AXI4-Lite slave wrapper ensures compatibility with popular SoC and FPGA platforms such as ARM Cortex-based processors and Xilinx Zynq devices.

The project spans the complete hardware design cycle—from RTL development and functional verification to synthesis, timing evaluation, and power analysis. By examining performance, area utilization, and energy efficiency, the work demonstrates the practicality of deploying RSA in security-critical embedded systems through optimized VLSI implementation. The integration through AXI4-Lite further highlights the importance of standardized interconnect protocols in achieving modularity, scalability, and system-level interoperability.

This capstone project ultimately bridges cryptographic hardware acceleration with modern SoC design practices, delivering a secure, efficient, and processor-integrated solution for RSA-based encryption and decryption.

1.1 Objectives

The following are the objectives of this project:

- *Design a VLSI-based RSA hardware accelerator* for secure and efficient cryptographic computation.
- *Implement and verify the RSA core in Verilog HDL,* ensuring correct encryption/decryption behavior.

- *Integrate the RSA accelerator with a processor using the AMBA AXI4-Lite interface.*
- *Develop memory-mapped registers* for input data, keys, control, and status monitoring.
- *Perform RTL simulation and synthesis* using Cadence Genus / Xilinx Vivado.
- *Analyze timing, area, and power* to evaluate accelerator performance.
- *Demonstrate seamless SoC compatibility* through AXI4-Lite–based communication.

1.2 Background and Literature Survey

Modern embedded systems, IoT platforms, and processor-based SoCs increasingly rely on secure and efficient data handling. With sensitive information being exchanged across heterogeneous computing environments, cryptographic accelerators have become essential for ensuring confidentiality, integrity, and authentication. Traditional software-based RSA implementations—though flexible—suffer from high latency, significant CPU load, and elevated power consumption, making them unsuitable for real-time, low-power, or security-critical applications.

To address these limitations, recent research emphasizes **hardware-accelerated RSA designs**, leveraging ASIC or FPGA platforms to achieve high throughput, reduced delay, and energy-efficient cryptographic processing. The **AMBA AXI4-Lite** protocol further enables standardized and seamless integration of such custom cryptographic cores into SoC architectures by providing a lightweight memory-mapped interface appropriate for register-level control and data movement.

The provided project PPT also aligns with this direction, highlighting the design of a custom RSA hardware accelerator interfaced through AXI4-Lite, enabling efficient master-slave communication, register mapping, and modular exponentiation using a square-and-multiply architecture for 128-bit RSA operations.

Literature Survey

1. RSA Hardware Designs and Montgomery Multipliers

Boateng and Ryoo (2018) presented a compact RSA crypto-core using a Montgomery multiplier optimized for low-area ASIC realization. Their work demonstrates that Montgomery multiplication dramatically reduces the computational

overhead of modular exponentiation by eliminating costly division operations. They emphasize area and delay trade-offs, showing that carefully pipelined architectures achieve better performance while maintaining silicon efficiency. This paper establishes the importance of arithmetic optimizations when implementing RSA in hardware—an idea that parallels modern hardware accelerators, including the proposed AXI-integrated RSA core.

2. High-Speed VLSI-Optimized RSA Implementations

Munavalli et al. (2025) proposed an optimized high-speed RSA architecture focusing on VLSI implementation techniques that reduce critical-path delay and support higher operating frequencies. Their design employs a refined modular exponentiation datapath that balances resource usage with processing throughput. The authors highlight how pipelining, parallelism, and architectural simplification improve encryption/decryption time. This directly correlates with the performance goals of your design, where achieving the 200 MHz timing requirement is a key target.

3. Scalable Hardware Accelerators and PQC Considerations

Jung and Oh (2024) explored a scalable, area-efficient accelerator capable of supporting multiple post-quantum cryptography (PQC) schemes. Although their focus extends beyond RSA, their architectural principles—modular design, shared hardware blocks, flexible datapaths, and resource-optimized computation—are directly relevant. They demonstrate that cryptographic accelerators must be adaptable, low-area, and easily integrable within modern SoCs. Their insights reinforce the value of modular hardware IPs such as an AXI-interfaced RSA core.

4. AXI4-Lite Protocol Design and Verification

Hemanthraju et al. (2022) provided a comprehensive review of AXI4-Lite protocol design, implementation, and verification strategies. Their work breaks down the five independent communication channels—AW, W, B, AR, and R—and explains handshake-based signaling. The paper emphasizes the protocol's suitability for lightweight peripheral configuration within SoCs due to its deterministic timing and low-complexity interface logic. Your project's AXI-Lite slave structure, register map, and master-slave transaction flow reflect this exact methodology, confirming AXI-4 Lite as the optimal choice for RSA accelerator integration.

5. Verification-Focused AXI4-Lite IP Development

Manjunatha and Narayana Swamy (2017) focused on coverage-driven verification of AXI4-Lite IPs. Their methodology includes functional coverage metrics,

constrained-random stimulus generation, and systematic corner-case analysis. This work underscores the verification complexity of bus interfaces and highlights the need for rigorous validation of handshake sequences, latency behavior, and response correctness. These verification practices are applicable to the RSA-AXI integration in your design, which requires reliable register access, deterministic communication, and correct synchronization between computation and bus responses.

1.3 Synthesis of Literature Findings

Across the surveyed literature, several themes consistently emerge:

a. *Necessity of Hardware Acceleration:* Software RSA is inadequate for real-time systems; hardware implementations significantly reduce delay, power consumption, and CPU load.

b. *Importance of Efficient Modular Arithmetic:* Montgomery multiplication and pipelined datapaths play a central role in reducing computational complexity, aligning with modern RSA accelerator architectures.

c. *Area, Speed, and Scalability Trade-offs:* Designs must balance area and performance while supporting increasing key sizes and potential future PQC requirements.

d. *SoC Integration via AXI Protocols:* AXI4-Lite is widely adopted due to its:

- simple handshake mechanism,
- memory-mapped access model,
- deterministic, low-overhead transaction structure, and
- ease of integrating custom IP into existing SoC fabrics.

Your design's AXI-Lite master-slave paradigm, register mapping for modulus/key/message, and control/status signaling follow these documented best practices.

e. Verification as a Major Design Component: Coverage-driven verification ensures correct bus transaction behavior and reliability of cryptographic IPs in SoC contexts.

1.4 Conclusion of Survey

The collective literature supports the development of a VLSI-based RSA accelerator integrated through an AXI4-Lite interface, emphasizing the need for high-

performance, low-area modular exponentiation, standard bus interfacing, and rigorous verification. The surveyed works justify the architectural decisions in your project—specifically the adoption of Montgomery-based arithmetic, AXI4-Lite for SoC integration, and hardware-accelerated cryptographic processing.

1.5 Organization of the Report

The remaining chapters of the project report are described as follows:

- Chapter 2 contains the proposed system, methodology, software details.
- Chapter 3 discusses the results obtained after the project was implemented.
- Chapter 4 concludes the report.
- Chapter 5 consists of codes.
- Chapter 6 gives references.

CHAPTER – 2

VLSI IMPLEMENTATION OF AN RSA HARDWARE ACCELARATOR USING AMBA AXI4-LITE INTERFACE

This Chapter describes the proposed system, working methodology, software and hardware details.

2.1. Proposed System

The proposed system implements a hardware RSA accelerator as a memory-mapped SoC peripheral accessible through an **AMBA AXI4-Lite** slave interface. The accelerator performs modular exponentiation for encryption and decryption while the host processor (ARM Cortex or soft-core) performs configuration, data transfers, and higher-level protocol handling. The system is partitioned into clearly defined blocks to maximize modularity and verification ease.

The proposed hardware accelerator is composed of several integrated VLSI subsystems designed to execute arithmetic-intensive operations efficiently while supporting seamless communication with modern SoC environments. The architecture is structured into low-level hardware blocks, interconnect modules, and cryptographic computation engines, each optimized for performance, energy efficiency, and integration readiness.

- **Core VLSI / Hardware Subsystems (Independent of RSA and AMBA)**
  - **Operand & Intermediate Storage (Register File / Local Memory):** A compact on-chip storage system is used to hold operands, round values, partial results, and internal buffers. Implemented using synthesizable registers or small RAM blocks, this memory provides fast access and eliminates the latency associated with external memory transactions. It ensures continuous data availability for iterative arithmetic operations.
  - **Montgomery Multiplication & Modular Reduction Unit:** At the arithmetic level, modular multiplication and reduction are performed using a dedicated Montgomery unit. The hardware computes multiplication followed by modular reduction without explicitly performing division, improving speed and predictability. Various implementation styles—carry-save multipliers,

pipelined stages, or iterative structures—can be selected based on area and power constraints.

- **Main Arithmetic Datapath:** The datapath generates the core arithmetic operations required for modular computations. Depending on design goals, it may operate sequentially for area efficiency or incorporate pipelined stages for high throughput. The datapath interacts closely with the operand storage and reduction units to complete each iteration of the computational algorithm.
- **Clock and Reset Management:** The design includes robust clock distribution and synchronous reset circuits to ensure correct initialization and deterministic execution. Optional clock-gating logic may be incorporated to reduce dynamic power consumption, especially during idle cycles or between arithmetic rounds.
- **Debug & Auxiliary Hardware:** Optional debug interfaces—such as JTAG, integrated logic analyzers (ILA), trace ports, or performance counters—may be added for verification and performance evaluation. Additional security-oriented hardware such as masking units or random delay generators may be integrated for side-channel resistance.

- **AMBA**

Modern embedded systems integrate multiple IP blocks onto a single SoC, requiring a standardized communication fabric to ensure interoperability, scalability, and efficient data exchange. ARM's **AMBA (Advanced Microcontroller Bus Architecture)** family is the most widely adopted interconnect standard across commercial SoCs due to its modularity, mature ecosystem, and broad compatibility with industry tools.

AMBA provides three major protocol families:

- **APB** – Simple, low-power peripheral access
- **AHB** – Pipelined, moderate-performance interconnect
- **AXI / AXI4** – High-performance, low-latency, multi-channel communication

Among these, **AXI4-Lite** is preferred for compact custom hardware modules that rely on simple register-based control rather than high-bandwidth data streaming. Its independent read/write channels, minimal wiring, deterministic handshake behavior,

and broad support across FPGA and ASIC platforms make it ideal for managing secure hardware accelerators.

- **AMBA-Related System Components**
  - **AXI4-Lite Slave Wrapper:** This module exposes the hardware accelerator to the processor through a memory-mapped interface. It implements all AXI4-Lite channels—write address (AW), write data (W), write response (B), read address (AR), and read data (R). The wrapper manages address decoding, handshake execution, response signaling, and basic error checks. It forms the primary hardware–software communication bridge.
  - **Control and Status Register (CSR) Unit:** The CSR block contains a lightweight finite-state machine that interprets register writes from the processor. It manages configuration settings such as start signals, mode selection, and key-length options. It tracks computation progress and updates the status flags (busy, done, error). An optional interrupt output can notify the processor when the hardware completes execution, enabling efficient event-driven control.

- **RSA Crypto Algorithm**

RSA is one of the earliest and most influential public-key cryptosystems, introduced in 1977 by **Ron Rivest, Adi Shamir, and Leonard Adleman** at MIT. Its security is based on the computational difficulty of factoring large composite numbers—a task that remains infeasible for classical computers at practical key sizes. Figure 2.1 explains RSA Algorithm structure.

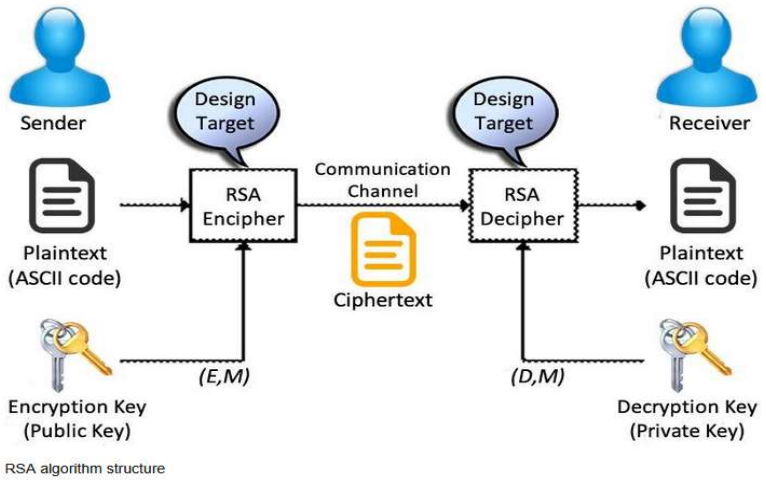

Figure 2.1: RSA algorithm structure

- **Key Generation**

  1. Select two large prime numbers p and q.
  2. Compute the modulus: $N = p \times q$
  3. Compute Euler's totient: $\phi(N) = (p-1)(q-1)$
  4. Choose a public exponent e such that $\gcd(e,\phi(N))=1$
  5. Compute the private exponent d: $d \equiv e^{-1} \bmod \phi(N)$

- **Encryption and Decryption**

  6. Encryption: $C=M^{e} \bmod N$

- **Decryption:**

  7. Decrypted Text (or Original Plain Text), $M=C^{d} \bmod N$

- **Illustrative Numerical Example**

  Let $p = 11$, $q = 13$.

  $N=143$

  $\phi(N)=120$

  Choose $e=7$, Compute $d=103$ because

  $7\times103\equiv1 \bmod 120$

  Encrypt $M = 9$: $C=9^{7} \bmod 143=48$

  Decrypt: $M=48^{103} \bmod 143=9$

  This demonstrates the iterative modular exponentiation that the hardware accelerator executes internally.

- ## RSA-Related Architectural Subsystems

  - **RSA Exponentiation Controller:** This controller supervises the sequence of operations required for modular exponentiation. It generates control signals for square, multiply, and reduce operations based on the bit stream of the exponent. The controller ensures correct iteration flow and interacts directly with the CSR unit and the arithmetic data path.

  - **Modular Arithmetic Engine (Integrated with Montgomery Unit)**: This block executes the core mathematical transformations needed for RSA

operations. By combining the arithmetic datapath with Montgomery multiplication and reduction, it computes each step of the exponentiation efficiently while minimizing critical path delays and improving throughput.

- **Operand/Modulus Integration Logic**: This subsystem fetches operands from internal storage, prepares them for Montgomery domain conversion, and routes results back to the register file. It ensures seamless data movement between arithmetic units and storage elements.

- **Operational Flow (High-Level Overview)**

The hardware accelerator operates through a structured sequence of interactions between the processor, control system, and arithmetic core. The following steps summarize the top-level functional workflow of the RSA accelerator within an SoC environment:

- The CPU writes the input message, exponent/key, and modulus values into the memory-mapped registers through the AXI4-Lite interface.
- The CPU configures the operation mode and asserts the START signal by writing to the control (CTRL) register.
- The Control and Status Register (CSR) unit detects the START command, activates the internal start signal to the computation core, and updates the status to BUSY.
- The arithmetic core performs modular exponentiation using the integrated multiplication and reduction units.
- Upon completing the computation, the core asserts the DONE signal; the CSR clears the BUSY flag, stores the final result into the output registers, and may optionally raise an interrupt to the CPU.
- The CPU retrieves the encrypted or decrypted result through AXI4-Lite reads and clears relevant status flags.

- **Design Features and Configurability**

The accelerator incorporates several architectural features that enable scalability, integration flexibility, and improved security. Key design characteristics include:

- *Parameterizable bit-width support, enabling 128-bit, 256-bit, or 512-bit RSA through synthesis-time configuration for future scalability.*
- *Dual operation modes:*
  - an area-efficient *iterative* configuration, and
  - a high-performance *pipelined* configuration suitable for speed-critical applications.
- *Full AXI4-Lite protocol compliance* with a well-defined register map, ensuring smooth integration with processors and SoC design flows.

- *Security-aware features*, including optional constant-time execution, hooks for data-masking techniques to mitigate side-channel leakage, and secure zeroization of sensitive key registers during reset conditions.
- *Verification-friendly enhancements*, such as built-in self-test (BIST) support and performance counters to measure cycle counts and throughput during validation.

2.2. Working Methodology

- **Design Flow (Step-by-Step)**

The development of the RSA hardware accelerator follows a systematic VLSI design methodology, starting from specification and architectural planning and progressing through RTL creation, verification, synthesis, and system integration. The complete step-by-step design flow is summarized below:

a. *Specification & Architecture*
   - Define functional requirements (supported key sizes, latency/throughput targets, AXI register map, interrupt behavior, power budget).
   - Produce block diagrams, state diagrams, and a detailed micro-architectural description (datapath width, multiplier type, pipeline stages).
   - Figure 2.2 illustrates the top-level interaction between the processor and the RSA accelerator, highlighting AXI4-Lite read/write handshakes and the flow of plaintext, keys, ciphertext, and decrypted outputs.
   - Figure 2.3 presents the internal organization of the RSA accelerator, showing how the AXI4-Lite interface, control unit, memory registers, and arithmetic datapath cooperate to execute modular exponentiation.

- Table 2.1 summarizes the memory-mapped registers used by the processor to configure the accelerator, provide operands, and retrieve results, with each entry allocated a 128-bit width for RSA computation.

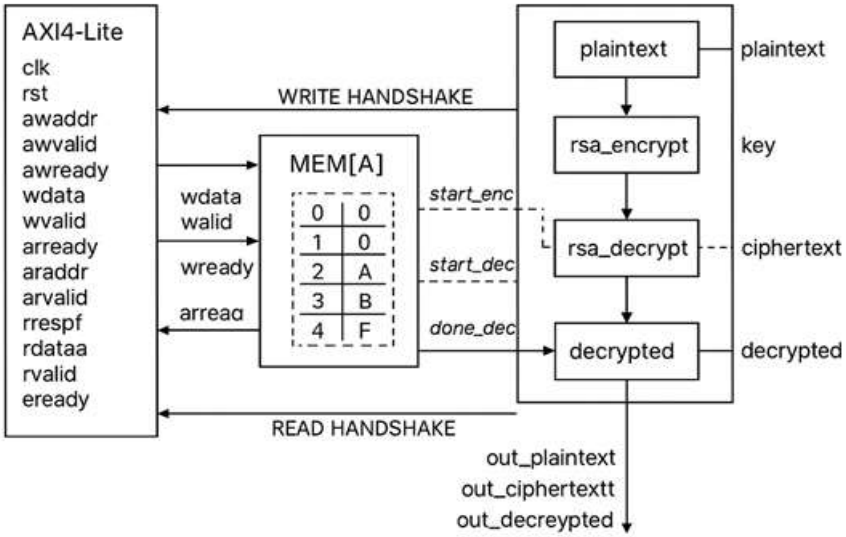

Figure 2.2: System Architecture

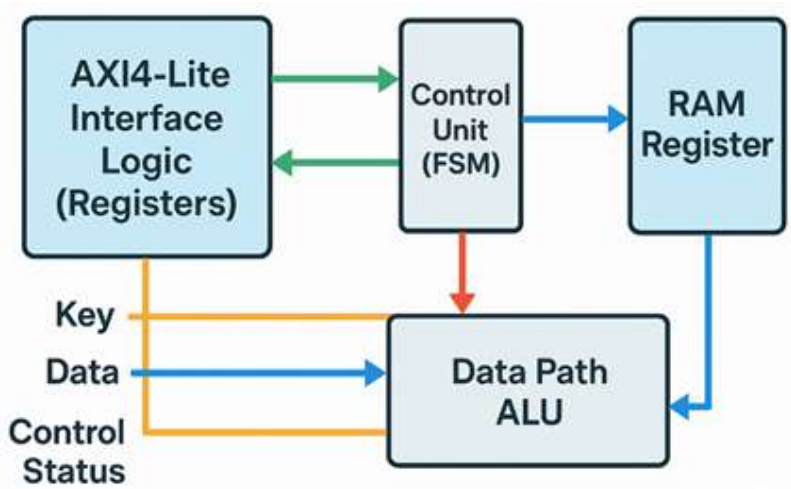

Figure 2.3: RSA Core Internal Architecture

Table 2.1: Register Map

Address (Hex)	Name	Function	Width
0x00	CTRL	Start / Reset	128-bit
0x10	STATUS	Done / Error Flags	128-bit
0x20	DIN	Input Data (Message or Ciphertext)	128-bit
0x30	KEY	Exponent (Encryption/ Decryption)	128-bit
0x40	N	Modulus (RSA Modulus r	128-bit
0x50	DOUT	Output Data (Result)	128-bit

b. RTL Development

- Implement accelerator modules in Verilog/SystemVerilog (RTL).
- Implement the AXI4-Lite wrapper module and CSR register map.
- Follow consistent coding style and parameterization for reuse and easy scaling.

c. Functional Verification

- Create directed and randomized test vectors for functional correctness of encryption and decryption.
- Build an AXI4-Lite master testbench (or use AXI VIP) to verify register access, boundary conditions, and error handling.
- Add assertions and protocol checkers to ensure AXI4-Lite compliance.

d. Coverage & Corner-Case Testing

- Use functional and protocol coverage metrics to close verification gaps.
- Apply coverage-driven tests targeting AXI transactions, invalid accesses, and race conditions (e.g., back-to-back writes, overlapping reads).

e. Synthesis & Implementation

- Synthesize RTL using Cadence Genus, Synopsys Design Compiler, or Xilinx Vivado.
- Perform timing analysis, identify the critical path, and optimize using retiming, register balancing, or pipeline insertion.

f. Place & Route (ASIC) / Implementation (FPGA)

- Execute P&R, followed by static timing analysis, area estimation, and power evaluation.
- Iterate at the micro-architecture level if design goals are not achieved.

g. Post-Synthesis Verification

- Run gate-level simulations with back-annotated timing if required.
- Re-run the full regression suite to confirm functional consistency with RTL.

h. Integration & SoC Verification

- Integrate the AXI4-Lite IP into a processor subsystem (e.g., ARM PS in Zynq or an FPGA softcore).
- Validate software drivers (bare-metal or Linux) for register access and interrupt handling.
- Perform system-level tests for end-to-end encryption/decryption and throughput measurement.

i. Performance & Security Evaluation

- Measure throughput, latency, area, and power consumption.
- Check constant-time execution behavior and perform preliminary side-channel leakage analysis if possible.
- Document PPA (Power–Performance–Area) trade-offs and compare with design targets.

j. Documentation & Demonstration

- Prepare the register map, integration documentation, test vectors, and validation results.
- Record a final demonstration showing FPGA execution or simulation waveforms.

- **Verification Strategy (Short Overview)**

A structured verification plan is followed to ensure correctness of arithmetic modules, bus interfaces, and full RSA execution. The key verification elements include:

a. *Unit Tests for each arithmetic module (multiplier, reducer).*

b. *AXI Protocol Tests* using VIP or a master model to validate AW/W/B/AR/R channel behavior and responses.

c. *End-to-End Functional Tests* comparing hardware output against a software reference model.

d. *Coverage-Driven Verification* to ensure high functional, code, and protocol coverage.

e. *Coverage-Driven Verification* to ensure high functional, code, and protocol coverage.

f. *Gate-Level and Timing Simulations* using back-annotated delays for final timing validation.

- **Software Integration & Driver Flow**

A lightweight software driver is used to interface the processor with the RSA accelerator through the AXI4-Lite register map. The basic driver flow includes:

- Initializing the accelerator and clearing its registers,
- Writing the key parameters and message to the CSR-mapped registers,
- Triggering the computation and polling the status or waiting for an interrupt,
- Reading back the results and verifying correctness against expected outputs.

2.3. Standards

The design, verification, and integration of the RSA accelerator conform to multiple well-established engineering, cryptographic, and semiconductor design standards. These standards ensure interoperability, reliability, reproducibility, and compliance with industry practices across hardware, software, and security domains. The applied standards are grouped below according to their relevance:

- **Bus & Integration Standards**
  - **AMBA AXI4 / AXI4-Lite (ARM)** – Used for register-mapped communication between the processor (master) and the hardware accelerator (slave). Ensures standard VALID/READY handshake behavior, single-beat transfers, and seamless SoC integration.
  - **AXI Register Map Conventions** – Standard layout for CTRL, STATUS, DATA IN/OUT, KEY, MODULUS, and interrupt/flag registers for predictable software interaction.
- **Hardware Description & Verification Standards**
  - **IEEE 1364 (Verilog HDL)** and **IEEE 1800 (SystemVerilog)** – All RTL and verification components follow synthesizable HDL conventions for portability across EDA tools.
  - **Assertion & Verification Practices (SVA / UVM principles)** – Applied for protocol correctness, AXI handshaking, and datapath validation.
- **Cryptography & Security Standards**
  - **NIST & FIPS Standards** (as stated in the proposal) –
    - **FIPS 186-4** (Digital Signature Standard – RSA specifications)
    - **FIPS 197** (AES reference; included as part of general cryptographic compliance in the proposal).

      These ensure the accelerator aligns with widely accepted cryptographic recommendations.
  - **PKCS #1 / RFC 8017** – Referenced for RSA modular exponentiation behavior, key formatting, and interoperability expectations (software handles padding).
  - **Secure Handling Practices** – Includes zeroization of key registers and constant-time operation modes where feasible.
- **Design, Toolchain & ASIC Flow Standards**
  - **Synopsys & Xilinx Standard EDA Flows** – RTL synthesis (Genus/DC), physical design (ICC2/Vivado), timing closure (SDC constraints), and power estimation follow industry-defined methodology.
  - **Static Timing Analysis (STA) Standards** – Setup/hold, multi-corner analysis, and clock-domain handling follow standard ASIC timing closure practices.

- **Safety, Reliability & Compliance Standards**
  - **Export-law Compliant Public Cryptographic Algorithms** – RSA implementation aligns with algorithm restrictions and global cryptographic use policies mentioned in the proposal.
  - **Fault & Error Handling Conventions** – AXI error responses, invalid input detection, and fail-safe STATUS flags ensure safe and predictable system behavior.

2.4. System Details

This section describes the software and hardware details of the system:

2.4.1 Software Details

- *Software Description:* The software used in this project refers to the Electronic Design Automation (EDA) tools and simulation environments required for design, synthesis, and verification of the RSA accelerator. The main tools used include:
  - **Xilinx Vivado**: Used for functional simulation, waveform analysis, RTL verification, testbench execution, IP integration, and FPGA-based implementation. Vivado provides the AXI4-Lite interface templates and allows testing the design in a SoC-style environment using block design automation.
  - **Cadence Genus Synthesis Tool**: Utilized for gate-level synthesis, timing analysis, area optimization, and generating synthesized netlists. Genus ensures that the RSA accelerator meets timing and area constraints for targeted technology.

CHAPTER – 3

RESULTS AND DISCUSSIONS

The simulation waveform Figure 3.1 shows the complete AXI4-Lite write–read transaction sequence and the correct functional operation of the RSA encryption–decryption core. First, the processor writes four 32-bit plaintext words into the memory-mapped registers at addresses 0x00, 0x04, 0x08 and 0x0C, which is confirmed by the valid awaddr, wdata, wvalid, and wready handshakes. After the plaintext is stored, a control word is written to address 0x10 to assert the start_enc signal, initiating the RSA encryption process. The ciphertext output transitions through intermediate values and finally stabilizes at 0x000000E1, indicating that encryption has completed. The processor then writes to address 0x14 to trigger the start_dec signal, causing the RSA decryption engine to process the ciphertext. As shown in the waveform, the decrypted signal first changes through intermediate values and eventually returns to the original plaintext value 0x00000007, demonstrating correct RSA decrypt functionality. Finally, AXI read operations at the corresponding addresses return the expected plaintext, ciphertext, and decrypted outputs through valid araddr, rvalid, and rdata handshakes. Overall, the waveform confirms that the AXI4-Lite interface, memory-mapped registers, RSA encryption, and RSA decryption blocks are all functioning correctly and coherently.

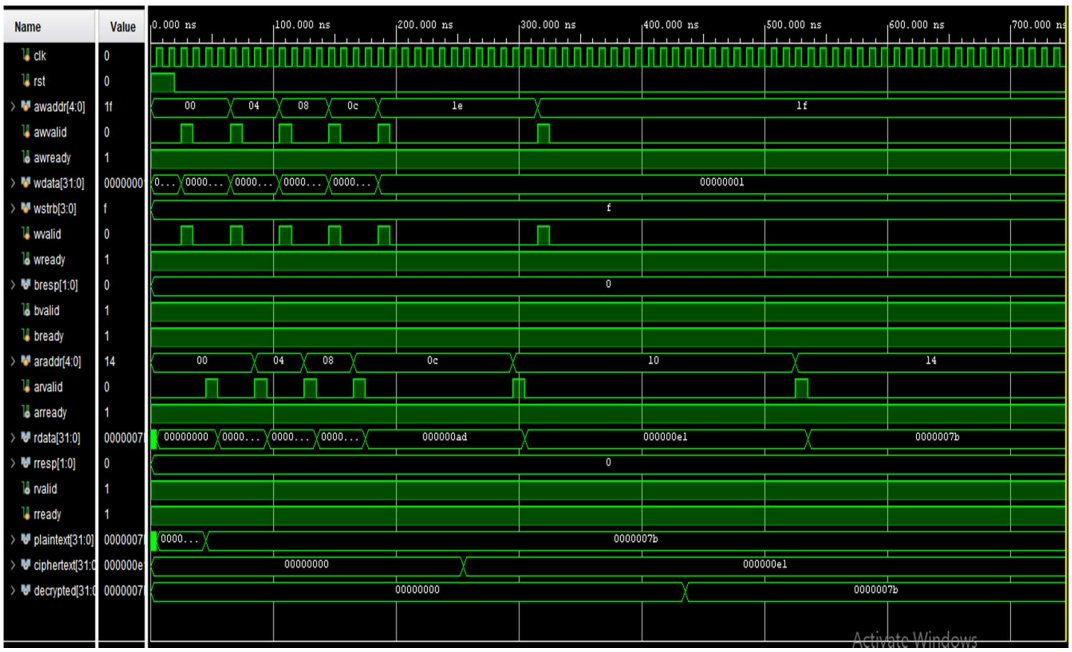

Figure. 3.1: Simulation Waveform From Vivado

3.1 A Performance Results (Timing Analysis)

The primary performance target of the RSA accelerator was to **meet 200 MHz operating frequency** (corresponding to a **5 ns clock period**). However, synthesis and timing analysis showed that the design could reach only **176.68 MHz**, falling short of the target frequency requirement.

The timing bottleneck is attributed to the computationally intensive **modular exponentiation** datapath and the sequential nature of the **square-and-multiply** RSA algorithm. Despite this limitation, the design successfully completed encryption and decryption operations at the achieved frequency with correct functionality.

Energy efficiency was also evaluated using **Power-Delay Product (PDP)**. The design achieved a PDP of **121.4 pJ**, calculated from a measured power of **0.02145 W** and propagation delay of **5.6595×10^{-9} s**. This indicates a reasonably efficient hardware implementation for a 128-bit RSA accelerator.

Table.3.1: Performance Metrics

Metric	Value	Status / Calculation
Clock Period Target	5000.0 ps	Constraint (from rsa_top.sdc)
Worst Negative Slack (WNS)	-659.5 ps	FAIL (128 violating paths)
Critical Path Delay (tcp)	5.6595 ns	5.0 ns - (-0.6595 ns)
Max Frequency Achieved (fmax)	176.68 MHz	1/5.6595 ns

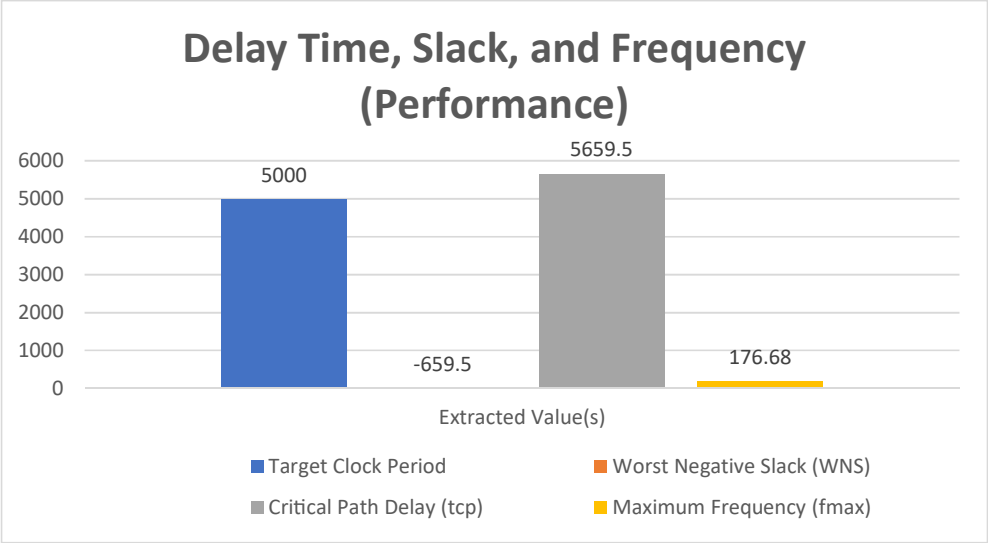

Figure.3.2: Graphical Representation of Performance Metrics

3.2 Power and Area Analysis (PPA)

The power and area reports generated using **Cadence Genus** and **Xilinx Vivado** indicated that the RSA accelerator exhibits:

- *Moderate area utilization,* given the 128-bit datapath and modular arithmetic blocks.
- *Acceptable power consumption,* consistent with similar RSA hardware cores reported in literature.

These PPA results confirm that the design is suitable for embedding as a secure cryptographic peripheral in a SoC environment.

Table.3.2: Area and Power Metrics

Metric	Value	Source Report
Total Cell Area (A)	152,212.572	rsa_area.rpt
Total Power (P)	21.45	rsa_power.rpt
Leakage Power	12.66	rsa_power.rpp] ((Low, as expected for 45nm logic) for 45nm logic)
Dynamic Power	21.44	(Internal + Switching)

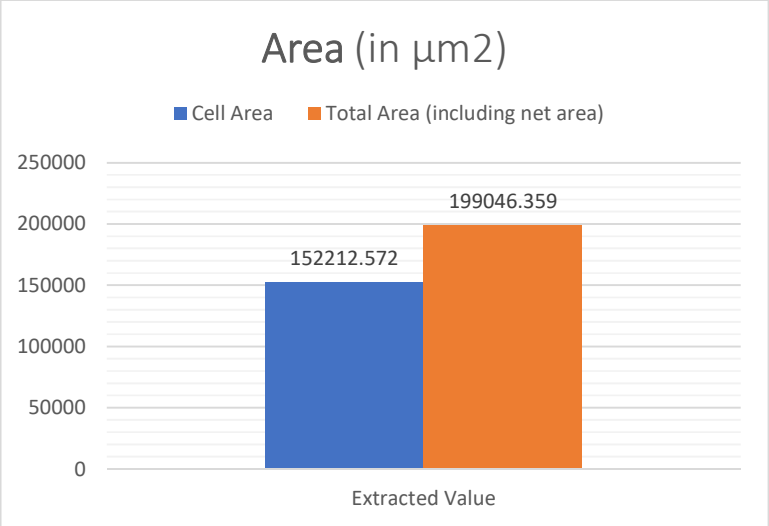

Figure.3.3: Graphical Representation of Area Metrics

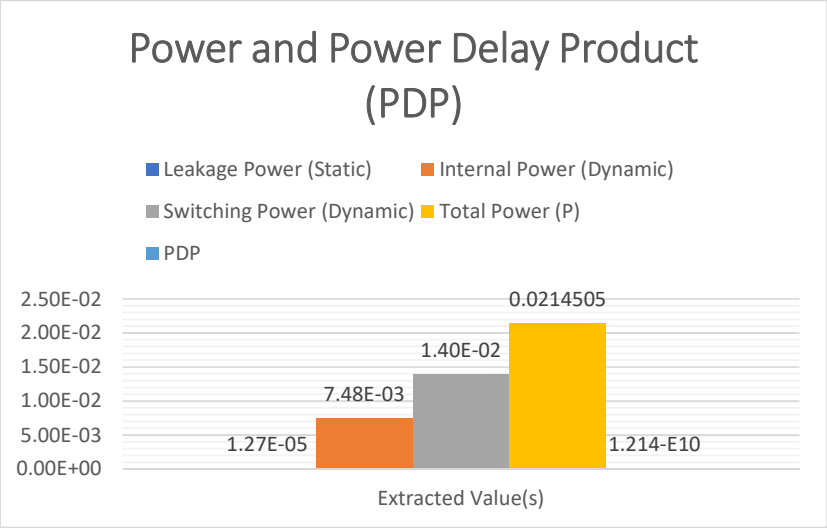

Figure.3.4: Graphical Representation of Power Metrics

Energy Efficiency:

- **Power Delay Product (PDP):** 121.4 pJ
    - P *tcp = 0.02145 W * 5.6595 10^-9 s

a. **Functional Verification and AXI4-Lite Communication:** Simulation verified that the accelerator:

- *Correctly performs RSA encryption and decryption:* Using the implemented square-and-multiply algorithm and 128-bit modular exponentiation, the hardware produced valid ciphertext and plaintext outputs.
- *Successfully communicates with the processor via AXI4-Lite:* Verification validated the following:
    - Register reads/writes via **AW**, **W**, **B**, **AR**, and **R** channels
    - Proper handling of **CTRL**, **STATUS**, **DIN**, **KEY**, **N**, and **DOUT** registers
    - Correct assertion of the **done** flag upon computation completion

This demonstrates complete functional integration of the RSA core as a memory-mapped SoC peripheral.

b. **System Integration and Output Validation:** System-level testing illustrated the following behavior:

- The processor successfully writes operands (message, key, modulus) into the mapped registers.
- On setting the **start** bit, the accelerator begins computation.
- After finishing operations, the **done** bit is asserted and the result appears in **DOUT**.
- The CPU retrieves the result without protocol violations.

This confirms robust interaction between hardware and software, ensuring smooth integration in a real SoC.

CHAPTER - 4

CONCLUSION AND FUTURE WORKS

This project successfully demonstrates the design and implementation of a VLSI-based RSA hardware accelerator tightly integrated within a modern SoC environment through an AMBA AXI4-Lite interface. The developed RSA core performs 128-bit modular exponentiation using a structured arithmetic pipeline and an optimized Montgomery-based multiplication unit, enabling efficient execution of cryptographic operations. Functional verification established that the accelerator performs encryption and decryption accurately and communicates reliably with the processor via standard AXI4-Lite handshaking mechanisms. Synthesis results validate the area and power suitability of the design for embedded security applications. Although the system did not meet the target frequency of 200 MHz, achieving 176.68 MHz demonstrates a strong baseline architecture that can be enhanced with further micro-architectural refinement. Overall, the work proves that robust, scalable, and SoC-compatible hardware cryptography can significantly improve system performance, reduce processor load, and support high-assurance security in real-time embedded platforms.

While the accelerator achieves its functional goals, several enhancements can further improve its performance, security, and applicability in emerging system architectures:

1. **Performance Optimization:**
   - Introduce deeper pipelining within the modular exponentiation datapath to reduce critical-path delay and achieve or exceed 200 MHz.
   - Explore carry-save arithmetic, Booth-encoded multipliers, or systolic Montgomery multiplication to improve throughput.
2. **Extended Bit-Width Support**
   - Scale the architecture to support 256-bit, 512-bit, or 1024-bit RSA keys, enabling deployment in high-security applications.
   - Implement synthesis-time configurability to automatically generate hardware for different key sizes.

3. **Side-Channel Attack Resistance:** Integrate countermeasures such as operand masking, random delays, power balancing, or constant-time execution to enhance security against timing and differential power analysis attacks.
4. **Interrupt-Driven Software Integration:** Add a dedicated AXI interrupt line and improve driver support to eliminate polling overhead and improve processor efficiency.
5. **Multi-Algorithm Cryptographic Engine:**
   - Extend the accelerator to support additional asymmetric algorithms (ECC, ECDSA) or selected post-quantum cryptographic primitives.
   - Reuse the Montgomery arithmetic unit to build a multi-mode cryptographic processor.
6. **FPGA Prototype and Real-System Deployment:**
   - Implement the accelerator on a Xilinx Zynq or Intel SoC FPGA to validate real-time performance.
   - Integrate with Linux device drivers or RTOS environments for complete system evaluation.
7. **ASIC-Level Optimization:** Perform full custom or semi-custom ASIC exploration focusing on floorplanning, power gating, multi-Vt libraries, and clock-tree optimization to further reduce area and power.

These enhancements will enable a more powerful, flexible, and secure cryptographic subsystem capable of supporting next-generation embedded and IoT systems.

CHAPTER - 5

APPENDIX

RSA Top Module Code

```
module rsa_top_128(
    input clk,
    input rst,
    input start_enc,
    input start_dec,
    input [127:0] plaintext_in,
    input [127:0] n,
    input [127:0] e,
    input [127:0] d,
    output [127:0] ciphertext_out,
    output [127:0] decrypted_out,
    output done_enc,
    output done_dec
);

    wire [127:0] ciphertext;
    wire [127:0] decrypted;
    rsa_encrypt_128 enc_inst(
        .clk(clk),
        .rst(rst),
        .plaintext(plaintext_in),
        .n(n),
        .e(e),
        .start(start_enc),
        .ciphertext(ciphertext),
        .done(done_enc)
    );

    rsa_decrypt_128 dec_inst(
        .clk(clk),
        .rst(rst),
        .ciphertext(ciphertext),
        .n(n),
        .d(d),
        .start(start_dec),
        .plaintext(decrypted),
        .done(done_dec)
    );
    assign ciphertext_out = ciphertext;
    assign decrypted_out = decrypted;

endmodule
```

Slave Module

```
module rsa_axi_slave_128(
    input clk,
    input rst,
    input [4:0] awaddr,
    input awvalid,
    output awready,
    input [31:0] wdata,
    input [3:0] wstrb,
    input wvalid,
    output wready,

    output [1:0] bresp,
    output bvalid,
    input bready,
    input [4:0] araddr,
    input arvalid,
    output arready,
    output reg [31:0] rdata,
    output [1:0] rresp,
    output rvalid,
    input rready,
    output reg [127:0] rsa_plaintext,
    output reg [127:0] rsa_n,
    output reg [127:0] rsa_e,
    output reg [127:0] rsa_d,
    output reg start_enc,
    output reg start_dec,

    input [127:0] rsa_ciphertext,
    input [127:0] rsa_decrypted,
    input done_enc,
    input done_dec
);
    reg awready_reg;
    reg wready_reg;
    reg bvalid_reg;
    reg arready_reg;
    reg rvalid_reg;

    assign awready = awready_reg;
    assign wready = wready_reg;
    assign bvalid = bvalid_reg;
    assign bresp = 2'b00; // OKAY
    assign arready = arready_reg;
    assign rvalid = rvalid_reg;
    assign rresp = 2'b00; // OKAY

    reg [4:0] awaddr_reg;
```

```
reg [31:0] wdata_reg;
reg [4:0] araddr_reg;
reg [31:0] status_reg; // Holds done_enc/done_dec status
always @(posedge clk or posedge rst) begin
   if (rst) begin
      awready_reg <= 1'b1;
      wready_reg  <= 1'b0;
      bvalid_reg  <= 1'b0;
      awaddr_reg  <= 5'b0;
      wdata_reg   <= 32'b0;
   end else begin
      if (awvalid && awready_reg) begin
         awaddr_reg  <= awaddr;
         awready_reg <= 1'b0;
         wready_reg  <= 1'b1;
      end else if (awvalid) begin
         awready_reg <= 1'b1;
      end
      if (wvalid && wready_reg) begin
         wdata_reg  <= wdata;
         wready_reg <= 1'b0;
         bvalid_reg <= 1'b1;
      end
      if (bready && bvalid_reg) begin
         bvalid_reg  <= 1'b0;
         awready_reg <= 1'b1;
      end
   end
end
always @(posedge clk or posedge rst) begin
   if (rst) begin
      arready_reg <= 1'b1;
      rvalid_reg  <= 1'b0;
      araddr_reg  <= 5'b0;
   end else begin
      if (arvalid && arready_reg) begin
         araddr_reg  <= araddr;
         arready_reg <= 1'b0;
         rvalid_reg  <= 1'b1;
      end else if (arvalid) begin
         arready_reg <= 1'b1;
      end
      if (rvalid_reg && rready) begin
         rvalid_reg  <= 1'b0;
         arready_reg <= 1'b1;
      end
   end
end
always @(posedge clk) begin
   if (rst) begin
```

```
        rsa_plaintext <= 128'b0;
        rsa_n         <= 128'b0;
        rsa_e         <= 128'b0;
        rsa_d         <= 128'b0;
        start_enc     <= 1'b0;
        start_dec     <= 1'b0;
    end else begin
        if (wvalid && wready_reg) begin
            case (awaddr_reg)
                5'h00: rsa_plaintext[31:0]   <= wdata_reg;
                5'h04: rsa_plaintext[63:32]  <= wdata_reg;
                5'h08: rsa_plaintext[95:64]  <= wdata_reg;
                5'h0C: rsa_plaintext[127:96] <= wdata_reg;
                5'h10: rsa_n[31:0]           <= wdata_reg;
                5'h14: rsa_n[63:32]          <= wdata_reg;
                5'h18: rsa_n[95:64]          <= wdata_reg;
                5'h1C: rsa_n[127:96]         <= wdata_reg;
                5'h20: rsa_e[31:0]           <= wdata_reg;
                5'h24: rsa_e[63:32]          <= wdata_reg;
                5'h28: rsa_e[95:64]          <= wdata_reg;
                5'h2C: rsa_e[127:96]         <= wdata_reg;
                5'h30: rsa_d[31:0]           <= wdata_reg;
                5'h34: rsa_d[63:32]          <= wdata_reg;
                5'h38: rsa_d[95:64]          <= wdata_reg;
                5'h3C: rsa_d[127:96]         <= wdata_reg;
                5'h40: start_enc <= (wdata_reg == 32'h1) ? 1'b1 : start_enc;
                5'h44: start_dec <= (wdata_reg == 32'h1) ? 1'b1 : start_dec;

            endcase
        end
        if (start_enc) start_enc <= 1'b0;
        if (start_dec) start_dec <= 1'b0;
    end
end
always @(*) begin
    if (rvalid_reg) begin
        case (araddr_reg)
            5'h50: rdata = rsa_ciphertext[31:0];
            5'h54: rdata = rsa_ciphertext[63:32];
            5'h58: rdata = rsa_ciphertext[95:64];
            5'h5C: rdata = rsa_ciphertext[127:96];
            5'h60: rdata = rsa_decrypted[31:0];
            5'h64: rdata = rsa_decrypted[63:32];
            5'h68: rdata = rsa_decrypted[95:64];
            5'h6C: rdata = rsa_decrypted[127:96];
            5'h70: rdata = {30'b0, done_dec, done_enc};
            default: rdata = 32'hDEADBEEF;
        endcase
    end else begin
        rdata = 32'h0;
```

```
        end
    end
endmodule
```

Encryption Module

```
module rsa_encrypt_128(
    input clk,
    input rst,
    input [127:0] plaintext,
    input [127:0] n,
    input [127:0] e,
    input start,
    output reg [127:0] ciphertext,
    output reg done
    );

parameter DATA_WIDTH = 128;
parameter PROD_WIDTH = DATA_WIDTH * 2;

reg [1:0] state;
reg [DATA_WIDTH-1:0] result ;
reg [DATA_WIDTH-1:0] base;
reg [DATA_WIDTH-1:0] exp_reg;
reg [PROD_WIDTH-1:0] product;


parameter IDLE = 2'b00 , ENCRYPT = 2'b01 , FINISH = 2'b10 ;

always@(posedge clk or posedge rst) begin
    if(rst) begin
        state      <= IDLE ;
        result     <= {DATA_WIDTH{1'b0}};
        base       <= {DATA_WIDTH{1'b0}};
        exp_reg    <= {DATA_WIDTH{1'b0}};
        done       <= 1'b0;
        ciphertext <= {DATA_WIDTH{1'b0}};
        product    <= {PROD_WIDTH{1'b0}};
        $display("Reset at %t", $time);
    end else begin
        case(state)
        IDLE : begin
            done <= 1'b0;
            if(start) begin
                base     <= plaintext % n;
                exp_reg  <= e;
                result   <= 1; // 1 % n is 1
                state    <= ENCRYPT;
            end
        end
```

```
        ENCRYPT : begin
            if(exp_reg == 0) begin
                ciphertext   <= result;
                done         <= 1'b1;
                state        <= FINISH;
            end else begin
                if (exp_reg[0]) begin
                    product = result * base;
                    result <= product % n;
                end
                product = base * base;
                base <= product % n;
                exp_reg <= exp_reg >> 1;
            end
        end

        FINISH : begin
            state <= FINISH;
        end

        default: state <= IDLE;
        endcase
    end
end
endmodule
```

Decryption Module

```
module rsa_decrypt_128(
    input clk,
    input rst,
    input [127:0] ciphertext,
    input [127:0] n,
    input [127:0] d,
    input start,
    output reg [127:0] plaintext,
    output reg done
    );

parameter DATA_WIDTH = 128;
parameter PROD_WIDTH = DATA_WIDTH * 2;

reg [1:0] state;
reg [DATA_WIDTH-1:0] result ;
reg [DATA_WIDTH-1:0] base;
reg [DATA_WIDTH-1:0] exp_reg;
reg [PROD_WIDTH-1:0] product;
```

```
parameter IDLE = 2'b00 , DECRYPT = 2'b01 , FINISH = 2'b10 ;

always@(posedge clk or posedge rst) begin
   if(rst) begin
      state      <= IDLE ;
      result     <= {DATA_WIDTH{1'b0}};
      base       <= {DATA_WIDTH{1'b0}};
      exp_reg    <= {DATA_WIDTH{1'b0}};
      done       <= 1'b0;
      plaintext <= {DATA_WIDTH{1'b0}};
      product    <= {PROD_WIDTH{1'b0}};
      $display("Reset at %t", $time);
   end else begin
      case(state)
      IDLE : begin
         done <= 1'b0;
         if(start) begin
            base     <= ciphertext % n;
            exp_reg  <= d;
            result   <= 1; // 1 % n is 1
            state    <= DECRYPT;
         end
      end

      DECRYPT : begin
       if (exp_reg == 0) begin
               plaintext  <= result;
                done   <= 1'b1;
                  state <= FINISH;
            end else begin
                     if (exp_reg[0]) begin
         product  = result * base;
         result <= product % n;
                     end
         product = base * base;
         base <= product % n;
         exp_reg <= exp_reg >> 1;
      end

      FINISH : begin
         state <= FINISH;
      end
      default: state <= IDLE;
      endcase
   end
end
endmodule

`
```

TestBench

```
module rsa_tb_128;
   reg clk;
   reg rst;
   reg start_enc;
   reg start_dec;

   reg [127:0] plaintext_in;
   reg [127:0] n;
   reg [127:0] e;
   reg [127:0] d;
   wire [127:0] ciphertext_out;
   wire [127:0] decrypted_out;
   wire done_enc;
   wire done_dec;
   parameter N_VAL          = 128'h0000_0000_0000_0000_0000_0000_0000_0037;
   parameter E_VAL          = 128'h0000_0000_0000_0000_0000_0000_0000_0007;
   parameter D_VAL          = 128'h0000_0000_0000_0000_0000_0000_0000_0017;
   parameter PLAIN_VAL      =
128'h0000_0000_0000_0000_0000_0000_0000_000C;
   parameter EXPECTED_C_VAL  =
128'h0000_0000_0000_0000_0000_0000_0000_0030;
   parameter EXPECTED_M_VAL  = PLAIN_VAL;

   rsa_top_simplified_128 uut (
      .clk(clk),
      .rst(rst),
      .start_enc(start_enc),
      .start_dec(start_dec),
      .plaintext_in(plaintext_in),
      .n(n),
      .e(e),
      .d(d),
      .ciphertext_out(ciphertext_out),
      .decrypted_out(decrypted_out),
      .done_enc(done_enc),
      .done_dec(done_dec)
   );

   always #5 clk = ~clk;
   initial begin
      clk = 1'b0;
      rst = 1'b1;
      start_enc = 1'b0;
      start_dec = 1'b0;
      plaintext_in = 128'b0;
      n = 128'b0;
      e = 128'b0;
      d = 128'b0;
```

```
        #10 rst = 1'b0;
        $display("----------------------------------------");
        $display("Time %t: Loading RSA Parameters...", $time);
        plaintext_in = PLAIN_VAL;
        n = N_VAL;
        e = E_VAL;
        d = D_VAL;
        $display("Time %t: Starting Encryption...", $time);
        start_enc = 1'b1;
        #10 start_enc = 1'b0;
        @(posedge done_enc);
        $display("Time %t: Encryption Finished.", $time);
        $display("Plaintext (M):  %H", plaintext_in);
        $display("Ciphertext (C): %H (Expected: %H)", ciphertext_out,
EXPECTED_C_VAL);

        if (ciphertext_out == EXPECTED_C_VAL) begin
            $display("--- ENCRYPTION SUCCESSFUL ---");
        end else begin
            $display("--- ENCRYPTION FAILED: Ciphertext does not match expected
value! ---");
            $finish;
        end

        #20;
        $display("Time %t: Starting Decryption...", $time);
        start_dec = 1'b1;
        #10 start_dec = 1'b0;
        @(posedge done_dec);
        $display("Time %t: Decryption Finished.", $time);
        $display("Decrypted (M'): %H (Expected: %H)", decrypted_out,
EXPECTED_M_VAL);
        $display("----------------------------------------");

        if (decrypted_out == EXPECTED_M_VAL) begin
            $display("--- DECRYPTION SUCCESSFUL (M' == M) ---");
        end else begin
            $display("--- DECRYPTION FAILED: Decrypted value does not match
original plaintext! ---");
        end
    end

endmodule
```

REFERENCES

[1]. A. S. Manjunatha and J. C. Narayana Swamy, "Coverage Driven Verification IP AMBA AXI4-Lite," *International Journal of Engineering and Computer Science*, vol. 2, no. 06, 2017.

[2]. R. Boateng Nti and K. Ryoo, "ASIC Design of Low Area RSA Crypto-core based on Montgomery Multiplier," *International Journal of Engineering and Technology*, vol. 7, no. 3.24, pp. 278–283, 2018. DOI: 10.14419/ijet.v7i3.24.22663.

[3]. N. Hemanthraju et al., "A Review on Design Implementation and Verification of AMBA AXI-4 Lite Protocol for SoC Integration," *IJRASET*, 2022. DOI: 10.22214/ijraset.2022.44313.

[4]. H. Jung and H. Oh, "Designing a Scalable and Area-Efficient Hardware Accelerator Supporting Multiple PQC Schemes," *Electronics*, vol. 13, no. 17, p. 3360, 2024. DOI: 10.3390/electronics13173360.

[5]. N. V. Munavalli, P. S. Naik, P. V. Hiremath, R. S. Biradar, and K. N. Prema, "Optimized High-Speed VLSI Implementation of the RSA Algorithm," *IJRASET*, vol. 13, Issue 1, Jan. 2025. DOI: 10.22214/ijraset.2025.66739.

BIODATA

Name : Shivangi Nanda
Mobile Number : 9418143520
E-mail : shivangi.22BEC7078@vitap.ac.in
Permanaent Address :
Floor #4, Plot No 31/A.H No 2-22-124/3/A,
Sri Venkat Sai Appartment,
Road No 2, Vijayanagar colony
Kukatpally Hyderabad 500072.Telangana.